

NYC Taxi Demand Prediction Using Machine Learning

Shiven Khurdi, Shreeti Shrestha

khurdi.s@northeastern.edu, shrestha.shre@northeastern.edu

Abstract

Urban transportation systems require accurate demand forecasting to optimize resource allocation and improve service quality. This project develops and compares multiple regression models to predict hourly taxi pickup demand across New York City zones. Using the NYC Taxi and Limousine Commission dataset containing approximately 2.9 million trips from January 2024, we implement and evaluate Linear Regression (baseline), Random Forest, XGBoost, a feed-forward Deep Neural Network (DNN), and LSTM for temporal pattern recognition. Our analysis reveals that XGBoost achieves the best performance with a test RMSE of 15.50 and R^2 of 0.958, outperforming both traditional methods and deep learning approaches. We find that well-engineered temporal features enable tree-based models to capture demand patterns more effectively than neural networks on this dataset.

1 Introduction

Accurate taxi demand prediction is crucial for efficient urban transportation management, affecting driver allocation, passenger wait times, and overall system efficiency. In cities like New York, where millions of taxi trips occur monthly, predicting demand patterns can significantly impact both service quality and driver earnings.

The primary challenge in taxi demand forecasting lies in capturing complex spatio-temporal dependencies: demand varies by location (business districts vs residential areas), time (rush hours vs late night), and day of week (weekdays vs weekends). While numerous studies have addressed demand prediction, comparing how different model architectures—from traditional machine learning to deep neural networks—handle these patterns remains valuable for understanding the trade-offs between model complexity and prediction accuracy.

This project systematically evaluates five approaches on their ability to predict hourly pickup counts across NYC’s taxi zones. We focus on practical considerations such as computational efficiency and interpretability alongside predictive performance. Our key contributions include:

- A comprehensive preprocessing pipeline that aggregates raw trip data into zone-level hourly demand with engineered temporal features
- Systematic comparison of linear, tree-based, and neural network models under consistent evaluation conditions
- Analysis of model performance across different demand levels and time periods

2 Related Work

Taxi demand prediction has been extensively studied using various approaches. Traditional statistical methods like ARIMA and its variants were early solutions for time-series forecasting of transportation demand [6]. However, these methods struggle to capture complex non-linear patterns and spatial dependencies.

Machine learning approaches have shown significant improvements. Gradient boosting methods, particularly XGBoost, have achieved strong results on tabular transportation data [2]. Recent work by Grinsztajn et al. [5] demonstrated that tree-based models consistently outperform deep learning on structured tabular data, which aligns with our problem setting.

Deep learning approaches have also been applied to taxi demand prediction. LSTMs and other recurrent architectures have been used to capture temporal dependencies [3]. More recently, graph neural networks have been proposed to model spatial relationships between zones [7]. However, these approaches typically require large datasets and may not outperform simpler methods when strong hand-crafted features are available.

Our work contributes a systematic comparison of traditional ML, gradient boosting, and deep learning methods on the same dataset with consistent preprocessing and evaluation, providing insights into when each approach is most effective.

3 Dataset

3.1 Data Source

We use the NYC Taxi and Limousine Commission (TLC) Yellow Taxi Trip Record dataset for January 2024, publicly available from the NYC TLC website [1]. The raw dataset contains approximately 2.9 million individual trip records with features including pickup/dropoff timestamps, locations (zone IDs 1-263), fare amounts, trip distances, and passenger counts.

3.2 Preprocessing

The raw data underwent several preprocessing steps:

Data Cleaning: We removed trips with invalid pickup location IDs (outside the valid range of 1-263), negative fare amounts, trip distances exceeding 100 miles, and timestamps outside January 2024. This removed approximately 49,000 records (1.7% of data).

Aggregation: Individual trips were aggregated into hourly counts by pickup zone, resulting in 76,171 zone-hour samples across 258 active zones and 744 hours (31 days $\times$ 24 hours).

Feature Engineering: We derived 21 features for each zone-hour sample:

- *Temporal features:* hour of day (0-23), day of week (0-6), weekend indicator, day of month, morning rush indicator (7-9 AM), evening rush indicator (5-8 PM)
- *Lag features:* pickup counts from 1, 2, 24, and 168 hours prior
- *Rolling features:* 3-hour and 24-hour moving averages and standard deviations

- *Zone statistics*: historical average fare, distance, demand, and demand standard deviation per zone

After dropping rows with missing lag values (first 24 hours per zone), the aggregated final dataset contains 38,509 samples. A complete feature correlation heatmap of variables can be found in Appendix 9.1 (Figure 4).

3.3 Train/Validation/Test Split

We performed a temporal split to prevent data leakage, ensuring the model is evaluated on future data it has never seen:

- Training set: 25,353 samples (January 2-22, 70%)
- Validation set: 6,494 samples (January 22-27, 15%)
- Test set: 6,662 samples (January 27-31, 15%)

3.4 Target Variable Analysis

The target variable (hourly pickup count) exhibits significant variation with mean of 40.1 pickups/hour, median of 5 pickups/hour, and range of 1-723 pickups/hour. The distribution is heavily right-skewed, with high-demand Manhattan zones accounting for most pickups while outer borough zones remain sparse. This skewness presents a challenge for model training and evaluation.

4 Methods

4.1 Linear Regression (Baseline)

We implemented linear regression with Ridge regularization and polynomial features as our baseline. Using scikit-learn's Pipeline, we applied StandardScaler followed by PolynomialFeatures (degree 2) and Ridge regression.

Hyperparameters tried:

- Model type: Linear, Ridge, Lasso
- Regularization strength α : 0.1, 1.0, 10.0
- Polynomial degree: 1, 2

Best configuration: Ridge with $\alpha = 1.0$ and degree=2, selected based on validation RMSE.

4.2 Random Forest

Random Forest regression was implemented using scikit-learn's RandomForestRegressor [4].

Hyperparameters tried:

- Number of estimators: 50, 100, 150, 200
- Maximum depth: 10, 15, 20, 25, None

- Minimum samples split: 2, 5, 10, 20
- Maximum features: sqrt, log2, 0.5

Best configuration: 200 estimators, max_depth=25, min_samples_split=2, max_features=sqrt.

4.3 XGBoost

XGBoost was implemented using the xgboost library [2].

Hyperparameters tried:

- Number of estimators: 100, 200, 300
- Maximum depth: 3, 5, 7
- Learning rate: 0.05, 0.1
- Subsample: 0.8
- Colsample_bytree: 0.8

Best configuration: 200 estimators, depth=7, learning_rate=0.05.

4.4 Deep Neural Network (DNN)

We implemented a feed-forward DNN with 3 hidden layers using PyTorch.

Architecture:

- Input layer: 21 features (standardized using StandardScaler)
- Hidden layers: $128 \rightarrow 64 \rightarrow 32$ neurons with ReLU activation
- Dropout: 0.1 between hidden layers for regularization
- Output layer: 1 neuron with linear activation

Training configuration:

- Optimizer: Adam (lr=0.001)
- Loss function: MSE
- Batch size: 256
- Early stopping patience: 15 epochs
- Learning rate scheduler: ReduceLROnPlateau (factor=0.5, patience=5)

4.5 LSTM

For sequential modeling, we implemented an LSTM network in PyTorch [3].

Architecture:

- Sequence length: 24 timesteps (representing 24 hours of history)
- LSTM: 2 layers, 64 hidden units, dropout=0.2
- Fully connected layers: $64 \rightarrow 32 \rightarrow 1$

Training configuration:

- Data sorted by zone and time to create valid sequences
- Optimizer: Adam (lr=0.001)
- Early stopping patience: 10 epochs
- Learning rate scheduler: ReduceLROnPlateau

5 Results

5.1 Overall Performance

Table 1: Model Performance Comparison on Test Set

Model	RMSE	R ²	MAE
Linear Regression (Baseline)	17.38	0.947	10.18
Random Forest	16.32	0.953	8.26
DNN	17.09	0.949	9.31
XGBoost	15.50	0.958	7.96
LSTM	33.99	0.798	14.61

XGBoost achieved the best performance across all metrics, with a 10.8% improvement in RMSE over the linear baseline. Random Forest came second with 6.1% improvement. DNN performed comparably to the baseline, while LSTM significantly underperformed with nearly double the RMSE of other methods. A comparison plot for model performance can be found at Appendix 9.2 (Figure 5).

5.2 Hyperparameter Search Results

Table 2: XGBoost Hyperparameter Search (Top 5 Configurations)

n_estimators	max_depth	learning_rate	Val RMSE	Val R ²
200	7	0.05	17.02	0.953
300	5	0.05	17.17	0.952
100	7	0.10	17.38	0.951
200	5	0.10	17.42	0.951
200	5	0.05	17.44	0.950

Model performance remains generally stable, where trees with depth 5-7 seem to produce the best results. Gradual boosting generalizes better, the dataset benefitting from slow, additive learning.

5.3 Performance by Demand Level

Table 3: XGBoost Performance by Zone Demand Level (Test Set)

Demand Level	RMSE	R ²	Samples
Low (bottom 33%)	1.64	-0.02	2,250
Medium (middle 33%)	9.52	0.90	2,264
High (top 33%)	25.43	0.93	2,148

All models struggle with low-demand zones even XGBoost, likely due to the inherent unpredictability of sparse events where counts = 0 - 5 per hour. High-demand zones show strong R² despite larger absolute errors, indicating the model captures demand patterns well in busy areas like Manhattan.

5.4 Peak Hour Performance

Table 4: Peak Hour Performance Comparison (Test Set, 7-9 AM & 5-8 PM)

Model	Peak RMSE	Overall RMSE
Linear Regression	17.59	17.38
Random Forest	16.10	16.32
DNN	17.65	17.09
XGBoost	14.94	15.50
LSTM	43.85	33.99

XGBoost maintains strong performance during peak hours when accurate predictions matter most for operational decisions. Interestingly, Random Forest performs slightly better during peak hours than overall, while DNN shows the opposite pattern.

5.5 Feature Importance

XGBoost feature importance analysis reveals that lag features dominate prediction:

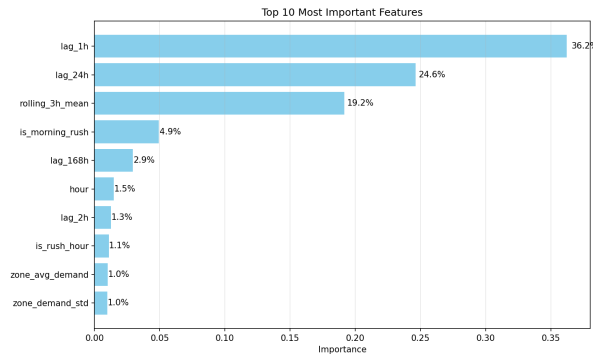


Figure 1: Top 10 Features by Importance for XGBoost

The top 3 features (lag_1h, lag_24h, rolling_3h_mean) account for 80% of total importance, indicating that recent historical demand and same-hour-yesterday patterns are the strongest predictors. This validates our feature engineering approach.

5.6 Prediction Visualizations

Figure 2 and 3 show predicted vs actual values and residual plots for the best (XGBoost) and worst (LSTM) performing models.

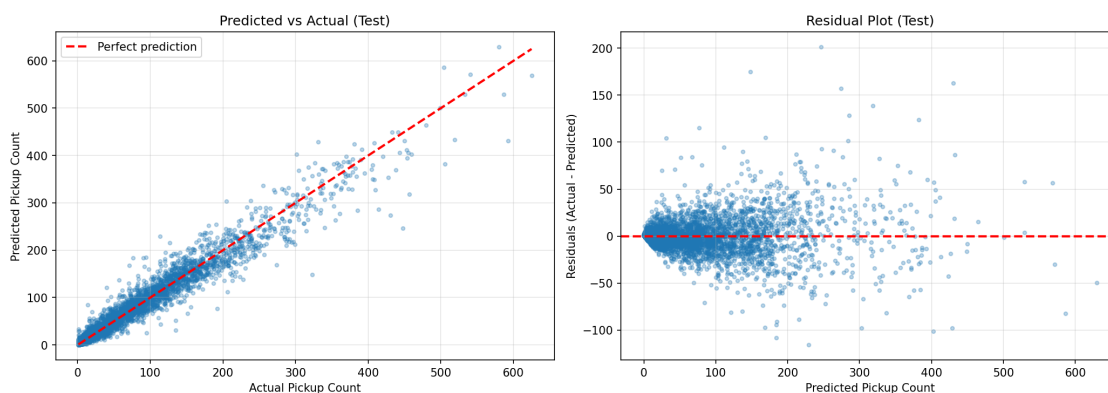


Figure 2: XGBoost (best performing model): Predicted vs Actual (left) and Residuals (right)

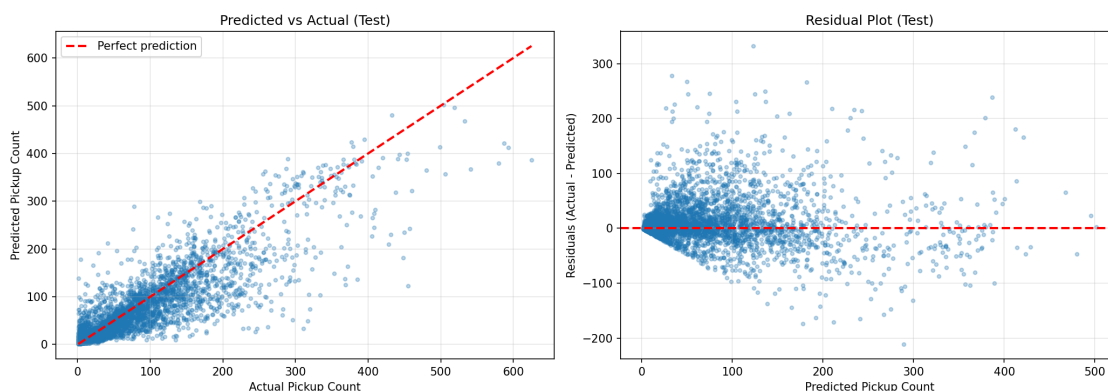


Figure 3: LSTM (worst performing model): Predicted vs Actual (left) and Residuals (right)

XGBoost shows tight clustering around the diagonal with residuals centered at zero. LSTM exhibits significant scatter and systematic underprediction for high-demand cases, confirming its poor performance.

6 Discussion

6.1 Why XGBoost Outperformed Other Methods

XGBoost achieved the best results across all metrics. We attribute this to several factors:

Effective Feature Engineering: Our lag and rolling features explicitly encode temporal patterns that neural networks would need to learn implicitly. XGBoost can directly leverage these hand-crafted features through its decision tree structure, splitting on the most informative features at each node.

Gradient Boosting Advantages: XGBoost’s sequential ensemble approach allows it to iteratively correct errors from previous trees, effectively handling the non-linear relationships in the data. The regularization (L1/L2) helps prevent overfitting despite the model’s flexibility.

Tabular Data Characteristics: Recent research [5] has shown that gradient boosting methods consistently outperform deep learning on structured tabular data with heterogeneous features, which matches our problem setting.

6.2 Why DNN Performed Comparably to Baseline

The DNN achieved similar performance to linear regression despite its greater complexity. This suggests that the polynomial features in the baseline already capture important non-linear interactions between variables. Additionally, with approximately 25,000 training samples, the DNN may not have enough data to learn patterns beyond what simpler models capture. The tabular nature of our features, with heterogeneous types and no inherent spatial structure, doesn’t play to DNN strengths which typically excel on image or sequential data with local patterns.

6.3 Why LSTM Underperformed

The LSTM significantly underperformed with test RMSE of 33.99 compared to 15.50 for XGBoost. Several factors contributed:

Sequence Construction Issues: Sorting data by zone to create sequences breaks natural temporal continuity across zones. The LSTM sees zone 1’s hours 1-24, then zone 2’s hours 1-24, rather than the natural city-wide temporal flow.

Feature Redundancy: Our lag features (lag_1h, lag_24h, etc.) already encode the sequential information that LSTM is designed to learn. This makes the recurrent architecture redundant—the LSTM is trying to learn patterns that are already explicitly provided as input features.

Limited Data: One month of data provides limited sequence diversity for learning long-term temporal patterns. LSTMs typically benefit from larger datasets with longer historical sequences.

High Target Variance: The large range of pickup counts (1-723) makes sequence-to-value prediction challenging, especially when consecutive timesteps can have very different values.

6.4 Limitations

Our study has several limitations worth noting. First, using a single month of data limits our ability to capture seasonal patterns and holiday effects that significantly impact taxi demand throughout the year. Second, we did not incorporate external features such as weather conditions, major events, or public transit disruptions, all of which are known to influence taxi usage. Third, our zone-level aggregation may miss finer spatial patterns, and graph-based approaches could potentially capture inter-zone relationships and spatial spillover effects.

6.5 Future Work

Several directions could extend this work. Expanding the dataset to a full year would enable the model to learn seasonal patterns and improve robustness. Incorporating weather data and event calendars could capture demand spikes during rain or major gatherings. Graph neural networks could model spatial dependencies between adjacent zones. Finally, attention-based architectures like Transformers could be explored as alternatives to LSTM for sequential modeling.

7 Conclusion

We compared five regression approaches for NYC taxi demand prediction: Linear Regression, Random Forest, XGBoost, DNN, and LSTM. XGBoost achieved the best performance (RMSE 15.50), followed by Random Forest (RMSE 16.32) and DNN (RMSE 17.09). LSTM significantly underperformed (RMSE 33.99), demonstrating that recurrent architectures do not automatically outperform tree-based methods on time-series problems when strong features are available.

Our results suggest that for time-series forecasting on structured data with limited samples, investing in feature engineering combined with gradient boosting yields better returns than complex neural architectures. The dominance of lag features in XGBoost’s importance rankings validates this approach. This finding aligns with recent research showing tree-based methods excel on tabular data.

Code Repository: <https://github.com/shivenkk/nyc-taxi-demand-prediction>

8 Team Contributions

Shiven Khurdi: XGBoost implementation and hyperparameter tuning, LSTM implementation and debugging, evaluation pipeline fixes for neural network models, report writing and analysis.

Shreeti Shrestha: Data preprocessing pipeline, feature engineering (temporal, lag, rolling, and zone features), Linear Regression baseline implementation, Random Forest implementation, DNN implementation, granular evaluation framework development.

Both team members contributed equally to experimental design, results analysis, and report editing.

References

- [1] NYC Taxi and Limousine Commission. *TLC Trip Record Data*. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- [4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [5] Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? *NeurIPS 2022*.
- [6] Moreira-Matias, L., Gama, J., Ferreira, M., Mendes-Moreira, J., & Damas, L. (2013). Predicting taxi-passenger demand using streaming data. *IEEE Transactions on Intelligent Transportation Systems*, 14(3), 1393-1402.
- [7] Yu, B., Yin, H., & Zhu, Z. (2018). Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. *IJCAI 2018*.

9 Appendix

9.1 Feature Correlation

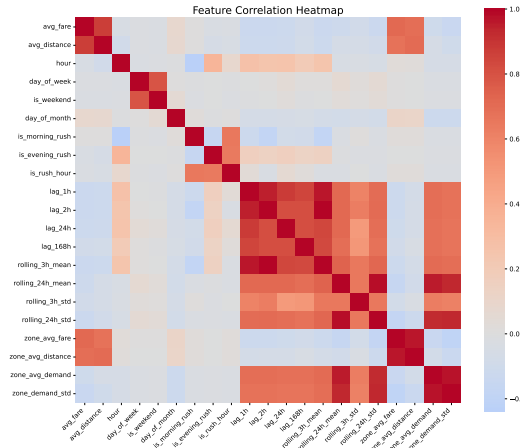


Figure 4: Feature correlation heatmap verifies domain knowledge (longer trip = more cost). It also shows how lag and rolling features dominate showing correlation of recent demand history (similar to the past hour).

9.2 Model Performance Comparison

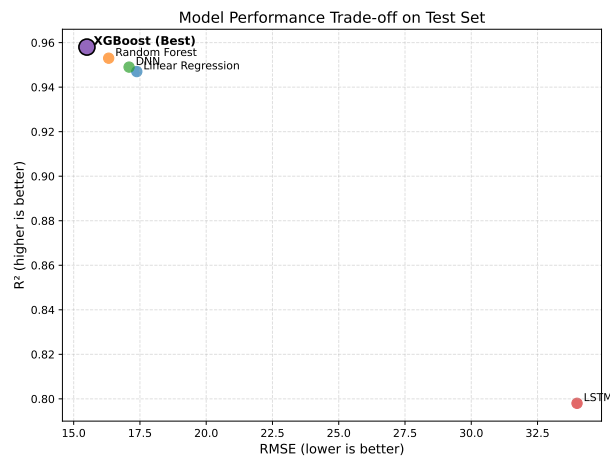


Figure 5: Performance of all 5 models. XGBoost has the greatest performance while LSTM performs poorly.